Which Parts of the "Modern Transformer Recipe" Actually Matter? A Controlled Ablation on TinyStories

Cohen Brunner Thomas

June 2026

Abstract

Contemporary language-model training stacks bundle a long list of small architectural and optimization tweaks, namely Muon, QK-norm, learnable QK-gain, LayerScale, value residuals, z-loss, and logit softcapping, and present them as a single "modern recipe." I disentangle this bundle with a controlled, multi-seed ablation. A fixed ∼97.6M-parameter decoder-only transformer is trained on TinyStories under an identical token budget (30.72M tokens, 1000 steps) across 22 configurations × 3 seeds = 66 runs, structured as a fractional factorial design (all-off and all-on anchors, single-feature add-ins, leave-one-out drops from the full recipe, and targeted 2 × 2 interaction cells). Two components dominate: switching AdamW → Muon and adding QK-norm each cut final validation loss by ≈0.13 nats, and the two are strongly *synergistic*. Together they buy ≈0.24 nats, far more than the sum of their individual effects. LayerScale and value residuals each contribute a smaller but real ≈0.05 nats and are mildly synergistic with each other. QK-gain and logit softcap are effectively free riders ($|\Delta| < 0.002$ nats), and z-loss is mildly harmful at this scale (+0.016 nats). The best single configuration is in fact the full Muon recipe *minus* z-loss (1.651 val loss), not the kitchen-sink "everything on" (1.669). All 66 runs trained to completion with zero divergences.

1 Introduction

The recipes shared in open LLM-training projects are cumulative: each new trick is added on top of the last, and the resulting stack is evaluated as a whole. This makes it hard to know which ingredients carry the gains, which are inert, and which are actively counterproductive at a given scale. It also makes the recipes hard to port. A practitioner copying the full stack inherits its dead weight and its failure modes.

This paper takes the seven toggles in the `TinyForge` training stack and measures each one in isolation and in combination, holding everything else fixed (model size, data, token budget, learning-rate schedule, weight decay, gradient clipping, batch order). The question is deliberately narrow and answerable: *under a fixed compute budget, what does each component contribute to validation loss, and do the components interact?*

2 Related work

The closest precedent is Narang et al. [9], who ran a large shared-setting sweep over many proposed Transformer modifications and found that most do not transfer: the gains often came from the original codebase rather than the idea, and few changes helped across tasks. I share their motivation, that recipe ingredients should be judged in a controlled setting rather than trusted by reputation, but differ in focus. Their study spans architectures and tasks at fixed scale to ask which modifications

generalize. This study fixes a single backbone, corpus, and token budget and instead resolves the *interactions* within one contemporary stack, isolating that the bulk of its advantage lives in a single Muon × QK-norm pair rather than spreading across the seven toggles. The components I test are all recent and postdate that survey, namely Muon [7], QK-norm [5], LayerScale [12], value residuals [14], and the z-loss and logit-softcap stabilizers of PaLM [1] and Gemma 2 [4]. Methodologically I adopt a fractional factorial design with matched on/off pairs and explicit 2×2 interaction cells, which lets a small run budget estimate both main effects and the targeted interactions that a one-factor-at-a-time sweep would miss.

This compute-constrained, single-GPU regime follows a line of work on small-scale language-model training, most directly Cramming [3], which re-examined the full pretraining pipeline under a one-day single-GPU budget, and the TinyStories corpus [2], whose low-entropy synthetic text makes $\sim 10^2$M-parameter models trainable to coherence in a short schedule. That regime is what lets me afford 66 runs, but it also bounds my claims: it is precisely the short-schedule, small-vocab setting in which stability components such as z-loss are expected to be inert, a caveat I return to in the discussion.

3 The components under study

All seven are independent toggles over a common backbone (12 layers, 12 heads, $d_{\text{model}} = 768$, SwiGLU MLP [10], RMSNorm pre-norm [13], RoPE [11], untied LM head). The backbone with all toggles off is the AdamW baseline [8].

Toggle	**What it does**	**Where**
optimizer	Muon [7] (Newton-Schulz-orthogonalized momentum, `match_rms_adamw` LR) on 2D body matrices + AdamW on embeddings/head/scales, vs. pure AdamW	optimizer
qk_norm	Per-head RMSNorm on Q and K before RoPE [5]	attention
qk_gain	Learnable per-head multiplicative gain folded into Q (scales scores pre-softmax)	attention
layerscale	Learnable per-channel residual gates on the attention and MLP branches (CaiT [12]), init 0.1	residual
value_residual	Each layer mixes its value with layer-0's value via a learnable per-layer λ (ResFormer [14])	attention
z_loss	`lse_square_scale` = 1e−4 regularizer on the log-partition [1]	loss
softcap	tanh logit softcap at 30.0	loss

z-loss and softcap are loss-side options of `LigerFusedLinearCrossEntropyLoss` [6]. The rest are model/optimizer changes. The Muon + AdamW split and weight-decay policy match the production `simple.py` stack.

4 Experimental setup

Model. ∼97.6M parameters (the small variation across configurations, 97,536,768 to 97,573,787, is the handful of extra scale/gate parameters some toggles add. It is <0.04% and does not confound the comparison).

Data. `roneneldan/TinyStories` [2], tokenized once with a fixed 8192-token byte-level BPE into a memory-mapped `uint16` corpus. Every run reads identical (x, y) blocks in identical order (no shuffling, deterministic batches), so the only differences between runs are the toggles and the seed.

Budget. 1000 optimizer steps, batch size 15, gradient accumulation 1, block size 2048 $\rightarrow$ 30,720 tokens/step $\rightarrow$ **30.72M tokens per run**. LR peak 1e$-$3, 99-step linear warmup then cosine decay to 0.1$\times$ peak, weight decay 0.1, gradient-norm clip 1.0, bf16 autocast, `torch.compile`. Validation loss is the mean cross-entropy over 100 held-out blocks at the final step.

Design. A full 2^7 grid (128 configs) was not run. Instead a fractional factorial focuses the budget on the questions that matter:

- **Anchors.** All-off (pure AdamW baseline) and all-on (full Muon recipe).
- **Add-one.** Baseline + exactly one toggle, for clean main effects of the features expected to matter on their own.
- **Leave-one-out.** Full recipe with exactly one toggle removed, to measure each component's *marginal* contribution inside the complete stack.
- **Targeted 2$\times$2 cells.** Muon$\times$QK-norm, LayerScale$\times$value-residual, z-loss$\times$softcap, z-loss$\times$QK-gain, and softcap$\times$QK-gain, for interaction terms.

22 unique configs $\times$ 3 seeds = 66 runs. Main effects are computed over matched on/off pairs (configs identical except the factor in question). Interactions are computed over matched 2×2 quadruples. Effects are reported as Δ(final val loss) = ON $-$ OFF, so **negative means the component helps**.

5 Results

5.1 Main effects

Factor	mean Δ	std	n pairs	verdict
qk_norm	-0.128	0.070	9	large help
optimizer (Muon)	-0.128	0.079	9	large help
value_residual	-0.053	0.032	9	moderate help
layerscale	-0.051	0.034	9	moderate help
qk_gain	-0.002	0.008	15	negligible
softcap	-0.000	0.010	15	negligible
z_loss	$+0.016$	0.008	15	mildly harmful

Two findings stand out. First, QK-norm and the Muon optimizer are the workhorses of the recipe, each worth $\approx$0.13 nats, an order of magnitude more than the next tier. Second, **z-loss consistently hurts** at this scale: its effect is positive in mean and tight in variance (std 0.008 over 15 pairs), so this is a real signal, not noise. z-loss is a numerical-stability regularizer designed for large-vocab, long-schedule training. With an 8192-token vocab and a 1000-step budget there is no instability to suppress (zero runs diverged), and the penalty simply pulls the model off the cross-entropy objective.

5.2 Interactions

interaction = [effect of A | B on] − [effect of A | B off]. **Negative = synergy** (A helps more when B is present), positive = redundancy/antagonism.

A	B	interaction	reading
optimizer	qk_norm	−0.158	strong synergy
layerscale	value_residual	−0.071	synergy
softcap	qk_gain	−0.007	∼none
z_loss	softcap	+0.006	∼none
z_loss	qk_gain	+0.003	∼none

The Muon×QK-norm interaction is the headline. The 2×2 cell means make it concrete:

	QK-norm off	QK-norm on
AdamW	1.987	1.929
Muon	1.964	**1.749**

Alone, Muon buys ≈0.023 nats and QK-norm ≈0.058 nats. Together they buy ≈0.238, roughly three times the sum of the parts. Muon orthogonalizes the body weight updates, and QK-norm bounds the scale of the query/key projections feeding attention. The data say these two stabilizations are complementary, and that **most of the "modern recipe" advantage is concentrated in this single pair.** LayerScale and value residuals show the same pattern more weakly: each gates or re-routes the residual stream, and they reinforce each other (−0.071).

5.3 Run rankings and the kitchen-sink trap

The best individual configurations are all Muon + QK-norm based:

rank	configuration	final val loss
1	Muon, all on **except z-loss**	**1.651** (seed mean 1.653)
4	Muon, full recipe (all on)	1.669 (seed mean 1.670)
19	Muon + QK-norm only	1.747
22	Muon, all on except QK-norm	1.781
n/a	AdamW baseline (all off)	1.987
n/a	AdamW full recipe	1.847

The top three runs (all three seeds) are the full recipe *with z-loss removed*, beating the kitchen-sink configuration by ≈0.018 nats, consistent with the main-effect sign of z-loss. Dropping QK-norm from the full recipe (rank 22, 1.781) is the single most damaging leave-one-out, again confirming QK-norm as the load-bearing component. Note also that the full recipe under **AdamW** (1.847) never reaches even the *worst* Muon recipe run, underscoring how much of the gain flows through the optimizer×QK-norm channel.

End to end, the recipe moves the model from 1.987 (baseline) to 1.651 (best), a 0.336-nat reduction, of which the Muon×QK-norm pair accounts for the large majority.

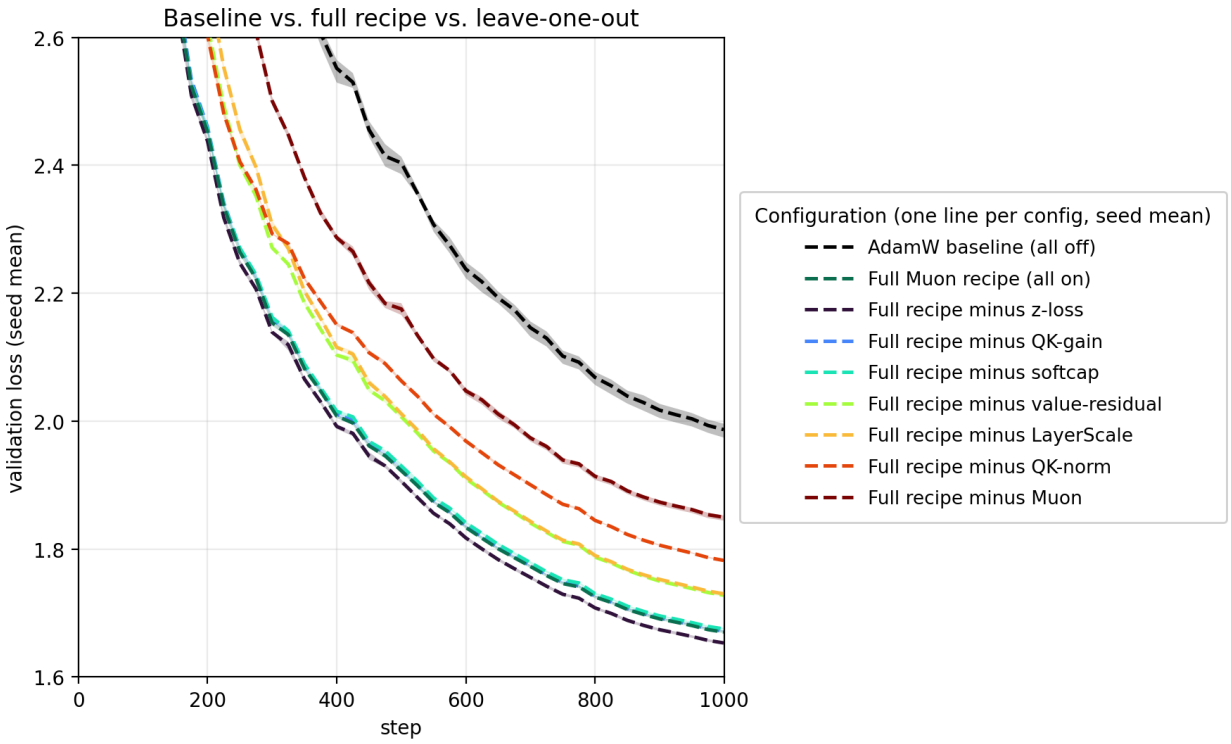

Figure 1: Validation-loss curves, one line per configuration (mean over 3 seeds, with a shaded min-to-max band that is narrow because seed spread, std ≈0.003 to 0.008 nats, is small next to the between-config gaps) for the two anchors (the all-off AdamW baseline, dashed black, and the all-on full Muon recipe, green) and the leave-one-out runs (full recipe with one component removed, labelled by the dropped component). The full recipe separates from the baseline early and holds the gap throughout. Among the leave-one-outs, dropping Muon or QK-norm is by far the most damaging, while dropping z-loss slightly *improves* on the full recipe.

5.4 Wall-clock cost

Each run logs total `wall_time_s`. Runs span 260 to 325 s (mean 278 s, 1000 steps), and the per-component cost, measured with the same matched on/off pairs as the loss effects, is concentrated in a single place.

Switching AdamW → Muon is the only clean cost signal, ≈20 s/run (+7.4%, std 0.66 over 9 pairs), the Newton-Schulz orthogonalization overhead on the body matrices. QK-norm's nominal +8.7 s has a std (7.1 s) larger than its mean, so its per-step elementwise-RMSNorm cost is effectively buried in shared-GPU timing jitter. Every other component is sub-1% and indistinguishable from noise. The cost ranking therefore agrees with the accuracy ranking: the two load-bearing components are also the only two with measurable overhead. The price is cheap for the payoff. The best configuration (298 s) costs +14.6% wall time over the baseline (260 s) for its −0.336-nat gain, and the Muon + QK-norm pair alone (292 s) buys −0.238 nats for ≈12% more time, with the remaining components adding ≈0.10 nats more at near-zero extra cost.

Factor	Δs	Δ%	std (s)	reading
optimizer (Muon)	+19.9	+7.4	0.66	real cost, tight
qk_norm	+8.7	+3.1	7.12	cost within noise
value_residual	+2.5	+0.9	10.09	noise
layerscale	+1.6	+0.6	10.45	noise
softcap	+0.3	+0.1	0.62	negligible
z_loss	−0.2	−0.1	0.50	negligible
qk_gain	−0.7	−0.3	7.35	noise

5.5 Does the verdict flip at larger vocab?

The main study's most fragile verdicts are the inert/harmful calls on z-loss and softcap. Both are output-softmax stabilizers expected to earn their keep only when the LM-head logits actually blow up, that is, at large vocab or long schedules. To test this directly I re-ran the z-loss×softcap 2×2 cell across a vocabulary sweep, holding everything else at a cheaper base regime (8 layers, 8 heads, $d_{\text{model}} = 512$, 1000 steps, block size 1024) and moving only the vocabulary: 8192 → 16384 → 32768. Each vocab uses its own freshly tokenized corpus, and 32768 is the top point, as the byte-level BPE saturates around 26.9k merges on TinyStories. Effects are the same ON−OFF Δ(final val loss) as before (negative = helps), each averaged over the two settings of the other factor.

vocab	params	z-loss Δ	softcap Δ	z-loss	softcap
8,192	34.1M	+0.0070	−0.0026	harmful	useful
16,384	42.5M	+0.0119	−0.0053	harmful	useful
32,768	59.3M	+0.0155	+0.0006	harmful	inert

The expected flip does not happen, and z-loss moves the wrong way. Rather than turning useful as the softmax grows, its penalty grows *monotonically more harmful* with vocab (+0.007 → +0.012 → +0.016 nats), tripling across a 4× vocabulary increase. Softcap, the gentler tanh clamp, is mildly useful at 8k to 16k but fades to inert by 32k. The z-loss×softcap interaction stays small and positive throughout (+0.004 → +0.010). Within the vocabulary range the corpus can support, then, neither stabilizer reverses its main-study verdict. The plausible reading is that on low-entropy TinyStories text the logits never approach the instability these regularizers are built for, so over this reachable range a larger vocab only enlarges the surface over which z-loss pulls the model off the cross-entropy objective. The verdicts of Section 5.1 are not an artifact of the 8k operating point. The schedule and model-scale axes (sketched in the sweep script but not run here) remain the open question.

6 Discussion

A practitioner's shortlist. If you must port only part of this stack, port Muon and QK-norm *together*. Separately they underwhelm, together they deliver. LayerScale and value residuals are a worthwhile second tier. QK-gain and softcap can be dropped with no measurable cost at this scale, and z-loss should be dropped outright unless a longer schedule or larger vocab reintroduces the instability it is meant to control.

Why z-loss flips sign. z-loss and softcap both exist to tame logit blow-up over long training. This study's regime (short budget, small vocab, zero divergences) is exactly the one where that

insurance is unnecessary, so it shows up as pure cost (z-loss) or pure no-op (softcap). The vocabulary sweep (Section 5.5) sharpens this. Pushing the vocab from 8k to 32k, the direction in which logit blow-up should worsen and the insurance should start to pay, does not rescue z-loss. Its penalty instead grows monotonically more harmful, and softcap only fades from mildly useful to inert. On low-entropy text the softmax simply never reaches the unstable regime these tools target. The result is a reminder that "stability" components should be judged against an actually-unstable baseline, not folded into a recipe by default.

Scope and limitations.

- **Scale.** One model size (∼98M), one corpus (TinyStories, simple, low-entropy text), one short budget (30.72M tokens). The *signs* of the large effects are likely robust. The inert/harmful verdicts for z-loss and softcap were the components most suspected of flipping at larger vocab. The vocabulary sweep in Section 5.5 tests this and finds they do not flip over the reachable range, and z-loss in fact worsens monotonically to vocab 32k. The schedule and model-scale axes remain untested and are the priority for a follow-up.
- **Fractional design.** Three- and higher-way interactions are not estimated. Main effects and the five targeted pairs are. The strong Muon×QK-norm synergy means single-factor numbers should be read as context-dependent, not additive.
- **Single quality metric.** Final validation loss only, with no downstream-task or generation-quality evaluation. Wall-clock cost is reported in Section 5.4, but on one GPU and software stack, so the absolute seconds are environment-specific even though the relative ordering is not.
- **Seeds.** Three seeds give a usable variance estimate. The per-factor std columns show the large effects sit well outside seed noise, while QK-gain and softcap sit inside it.

7 Conclusion

The "modern transformer recipe" is not monolithic. On a controlled TinyStories ablation, its benefit concentrates almost entirely in **Muon + QK-norm acting together**, with a smaller LayerScale + value-residual contribution and a trailing set of components that are inert (QK-gain, softcap) or mildly counterproductive (z-loss) at this scale. The single best configuration is the full recipe with z-loss removed. The broader lesson is methodological: stack ingredients earn their place against matched, multi-seed pairs and explicit interaction tests, not by accumulation.

References

[1] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

[2] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[3] Jonas Geiping and Tom Goldstein. Cramming: Training a language model on a single gpu in one day. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 11117–11143, 2023.

[4] Gemma Team, Google DeepMind. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[5] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4246–4253, 2020.

[6] Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. Liger kernel: Efficient triton kernels for llm training. *arXiv preprint arXiv:2410.10989*, 2024.

[7] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

[8] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[9] Sharan Narang, Hyung Won Chung, Yi Tay, Liam Fedus, Thibault Fevry, Michael Matena, Karishma Malkan, Noah Fiedel, Noam Shazeer, Zhenzhong Lan, Yanqi Zhou, Wei Li, Nan Ding, Jake Marcus, Adam Roberts, and Colin Raffel. Do transformer modifications transfer across implementations and applications? In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[10] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[11] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[12] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 32–42, 2021.

[13] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[14] Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. *arXiv preprint arXiv:2410.17897*, 2024.

A Reproduction

```
python DataScripts/prepare_data.py          # one-time tokenization
python DataScripts/run_matrix.py --seeds 3  # 66 runs, resumable
python DataScripts/analyze_results.py       # tables + figures
python DataScripts/flip_sweep.py --axis vocab --auto-prepare  # vocab sweep
python DataScripts/flip_analyze.py          # z-loss/softcap flip table
```

Per-run artifacts: `DataOutput/runs/<run_id>/{config.json,log.jsonl,summary.json}`. Aggregated tables and loss-curve figures: `DataOutput/analysis/{results.csv,main_effects.csv,interactions.csv,summary.md,loss_curves*.png}`. Vocab-sweep cells and their effects: `DataOutput/flip_runs/` and `DataOutput/analysis/flip_effects.csv`. All 66 main runs completed, 0 diverged.